2.

Let n be the number waiting (0 to 200), and let a = 0 mean wait and a = 1 mean dispatch. Dispatch happens before holding costs and new arrivals. One shuttle can be sent each period.

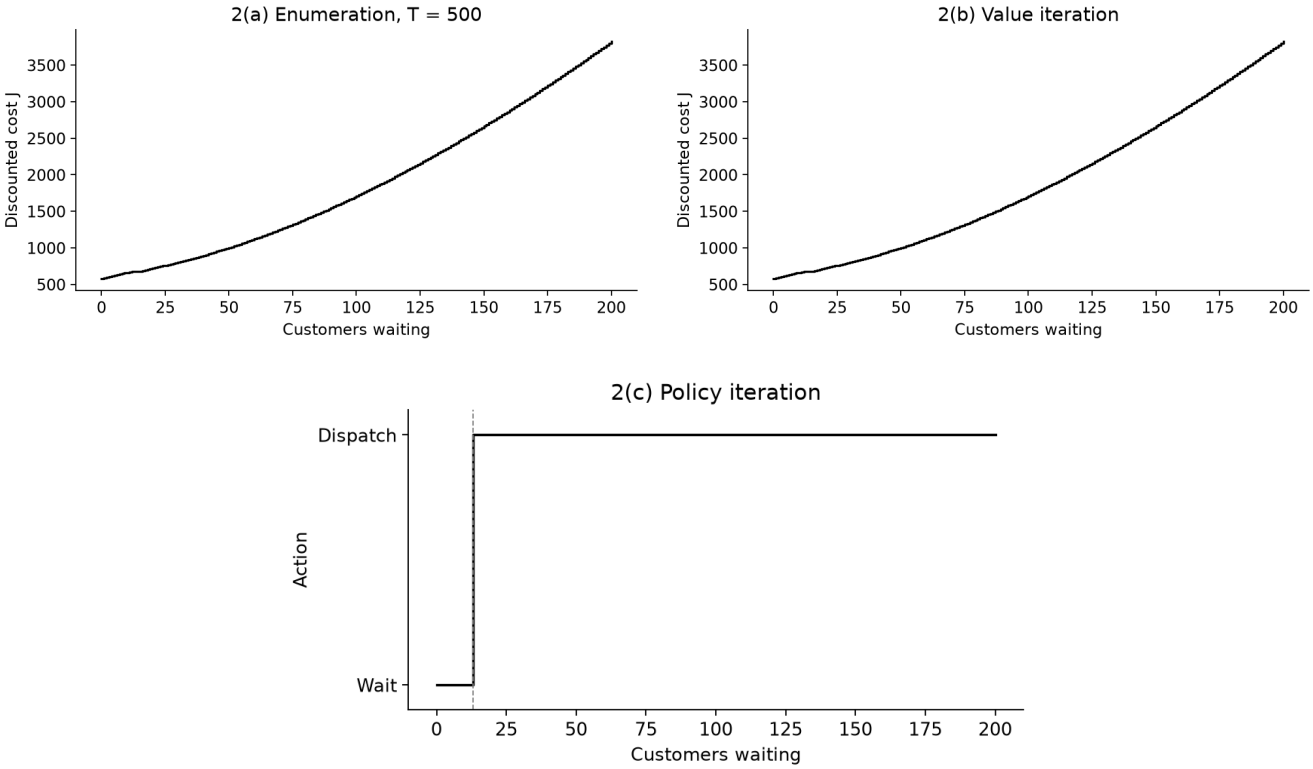
```
q = max(n - 15a, 0);  c(n,a) = 100a + 2q
n_next = min(200, q + A),  A uniform on {1,2,3,4,5}
J_t(n) = min_a [c(n,a) + 0.95 E J_(t+1)(n_next)]
```

J is discounted cost; reward value is V = -J. Excess arrivals are blocked without an extra penalty. For T=500, the terminal cost is zero.

2(a). Enumerate all states, actions and arrivals by backward induction for 500 periods.

2(b). Start value iteration at zero and stop when the largest change is below 1e-10.

2(c). Alternate exact policy evaluation, (I - gamma P)J = c, and greedy improvement.



Result: dispatch when n is at least 13; otherwise wait. J(0)=576.670, J(15)=676.670, and J(200)=3821.586. VI and PI give the same policy, with maximum cost difference 1.87e-9. The T=500 and infinite-horizon values differ by at most 4.94e-9.

Edge cases and checks: max(n-15,0) handles partial loads. At n=200, waiting keeps the queue at 200; dispatching gives next states 186-190. Empty queues and ties choose wait. With gamma=0, dispatch costs 100 but saves at most 30 immediately, so all states wait and J(n)=2n exactly. Free dispatch serves every nonempty queue; free holding gives J=0 and waiting everywhere. Both match the computed results.

3. Part 1

3(a). CartPole uses a 4-input network with 32 tanh units and 2 outputs. I train for 800 episodes with Adam, learning rate 0.01 and gamma=0.95, using one update per episode and discounted reward-to-go.

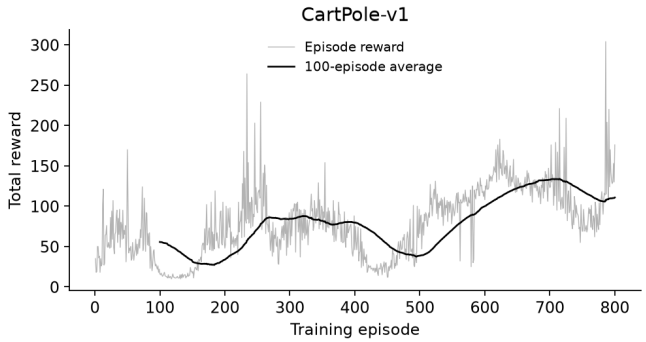
```
G_t = sum_(k=t)^(T-1) gamma^(k-t) r_k
CartPole loss = -sum_t gamma^t log pi(a_t|s_t) G_t
```

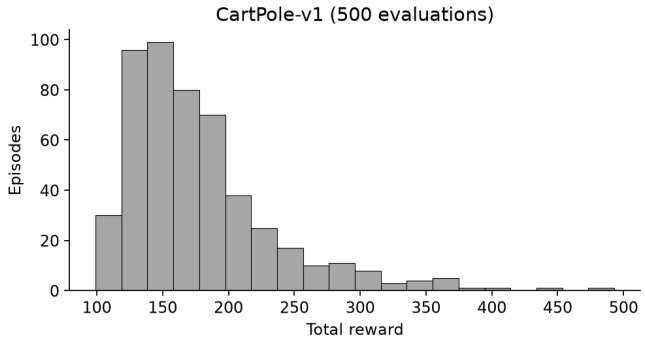
3(b). Pong uses 6400 inputs, 256 ReLU units and one Bernoulli logit for actions 2 and 3. I train for 4000 episodes (3000 initial + 1000 continued), updating after every 4 games with Adam, learning rate 0.001 and gamma=0.99. The smaller learning rate is used for stability. Warm-start training restores the policy, Adam state and training RNG, but restarts the environment without restoring ALE state. Frames use the supplied 80x80 binary preprocessing and consecutive-frame differences; the first difference is zero. Frame skip is 4 and sticky-action probability is 0.25.

For Pong, I truncate reward-to-go at the next scoring point and omit the outer time discount in the policy loss. This is a practical approximation to the full discounted episodic objective. Entire games are still used for training scores and evaluation.

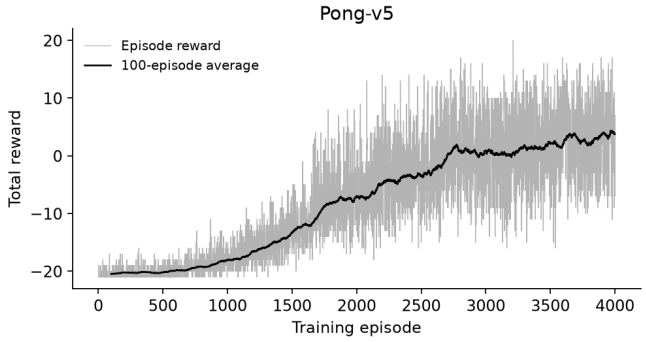
```
Pong loss = -(1/4) sum_games sum_t log pi(a_t|s_t) G_t(point)
```

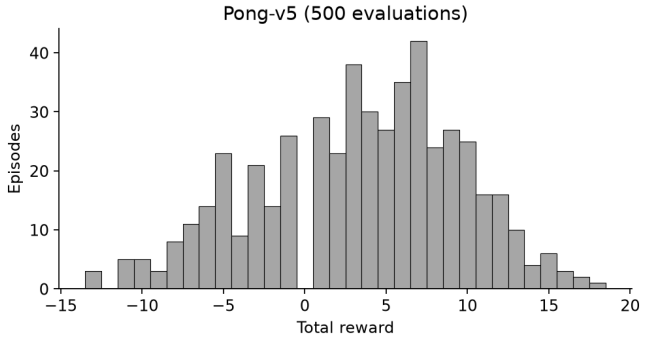
3(a).





3(b).





Environment / training episodes	Eval mean	Eval SD
CartPole-v1 / 800	176.82	55.99
Pong-v5 / 4000	3.43	6.23

Both training plots include the trailing 100-episode mean. Each final stochastic policy is evaluated for 500 full games without updates; SD uses denominator 500. Pong averages 3.43 points and wins 358/500 games (71.6%). CartPole uses training seed 7 and evaluation seeds 20000-20499. Pong uses initial training seed 1, continuation environment seed 70001 and evaluation seed 54321.

Checks and edge cases: termination or truncation ends a rollout, and the final reward is retained. Frame history resets each game. Pong returns reset at each score; zero-reward steps inherit the discounted next score. All 4000 Pong training games and 500 evaluations terminated normally, with no truncation and no evaluation parameter changes.

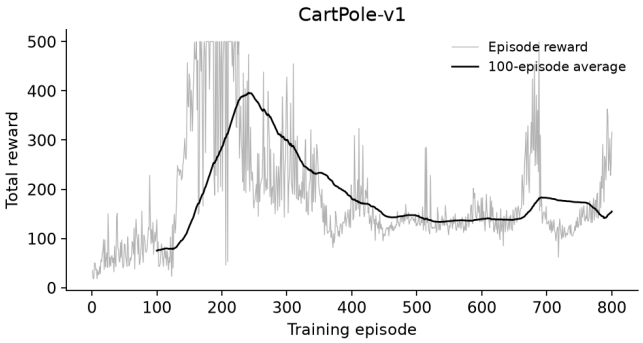
Policy Gradient: Part 2

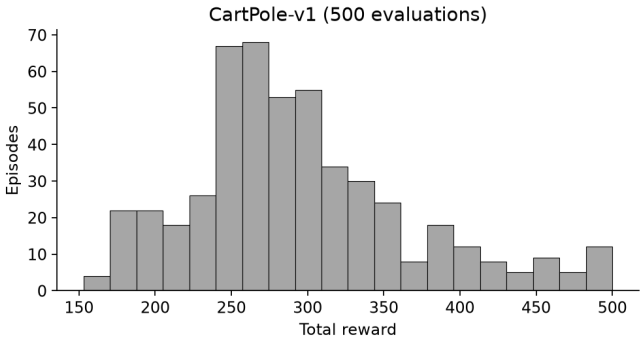
I use an exponential moving average (EMA) baseline for both environments. It starts at zero and stays fixed during an episode. After the policy update, b is replaced by 0.9b + 0.1 mean(G_t), using previous episodes without depending on the current action.

These baseline runs use 32 hidden units (tanh for CartPole, ReLU for Pong), two policy outputs, full-episode reward-to-go and one update per episode. CartPole uses 800 episodes, gamma=0.95 and learning rate 0.01; Pong uses 250 episodes, gamma=0.99 and learning rate 0.001. Training seed is 7 and evaluation seeds are 20000-20499. These retain the original baseline setup; they are not a matched comparison with the Part 1 Pong configuration.

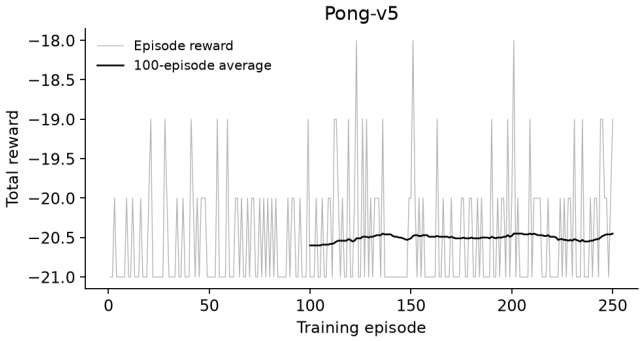
```
Loss = -sum_t gamma^t log pi(a_t|s_t) (G_t - b)
```

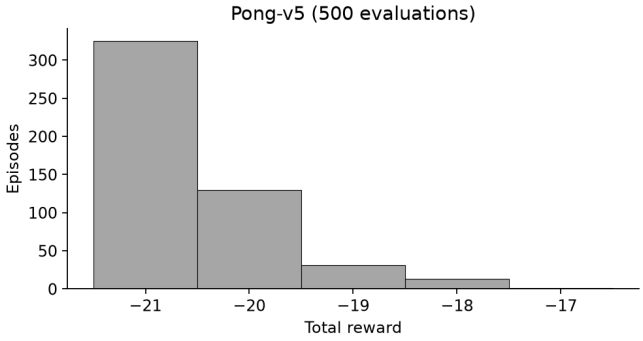
(a).





(b).





Environment / training episodes	Eval mean	Eval SD
CartPole-v1 / 800	294.39	72.46
Pong-v5 / 250	-20.53	0.74

Each policy is frozen for 500 evaluation episodes. The CartPole baseline policy balances for 294.39 steps on average. The Pong baseline policy averages -20.53 points and is not competitive.

Variance check: for each final baseline policy, I use the same 20 evaluation trajectories to compute gradients with b=0 and with the final EMA. The empirical covariance trace changes from 7823.25 to 50.41 for CartPole and from 418.84 to 116.25 for Pong. This directly checks variance reduction on fixed trajectories.

Edge cases: the first episode uses b=0. Updating the EMA only after the optimizer step avoids fitting the baseline to the current episode. Zero and negative rewards do not require division by a reward standard deviation. A constant baseline is unbiased, but a poor choice can increase variance.

2. Validation: I checked a deterministic problem with capacity 3, cap 8, arrivals 2, dispatch cost 5, holding cost 1, gamma=0.8 and horizon 4. Enumerating all 16 action sequences from each of 9 states agrees with dynamic programming to 3.55e-15. With no arrivals and n=0,...,15, the analytic cost min(40n,100) matches the algorithm to 1.89e-9.

Policy Gradient, Parts 1 and 2. Validation: rewards [1,2,3] with gamma=0.5 give full returns [2.75,3.5,3]. For Pong, rewards [0,1,0,-1] with gamma=0.5 give point-truncated returns [0.5,1,-0.5,-1]. Both match hand calculation. In a one-step bandit with P(a=1)=0.3 and reward=a, exact enumeration gives gradient 0.21 with or without baseline 0.3. Its variance falls from 0.1029 to 0.0336 (67.3%). This tests the estimator independently of game performance. All reported evaluations leave model parameters unchanged.

Reproduce: use Python 3.12 and run these commands from the repository root. The selected Pong model and scores are in results/pong_selected. The two Pong commands reproduce the initial training and warm-start continuation; use fresh output paths. The README also gives a shortcut using the supplied initial checkpoint. build_report.py uses saved results and does not retrain.

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python shuttle.py
python policy_gradient.py --task cartpole --episodes 800 \
  --learning-rate 0.01 --output results/cartpole_plain
python policy_gradient.py --task cartpole --baseline --episodes 800 \
  --learning-rate 0.01 --output results/cartpole_baseline
python pong_tuning.py --recipe ishan --episodes 3000 --seed 1 \
  --output results/pong_initial_rerun
python pong_continue.py --source results/pong_initial_rerun/policy.pt \
  --additional-episodes 1000 --train-env-seed 70001 --eval-seed 54321 \
  --output results/pong_rerun
python policy_gradient.py --task pong --baseline --episodes 250 \
  --learning-rate 0.001 --output results/pong_baseline
python policy_gradient.py --validate --output results/policy_validation.json
python -m pytest -q
python build_report.py
```

shuttle.py runs the deterministic and exhaustive checks. The --validate command checks full returns, the bandit and random policies. The tests also verify point-truncated returns and the four-game update. Training commands save raw scores, metrics, models and plots, followed by 500 frozen evaluations. The baseline runner additionally saves the 20-trajectory variance check and first-10-step trace.

With three more months: I would repeat training over multiple seeds, test longer Pong training, use identical configurations for baseline ablations, measure gradient variance over more trajectories, and test shuttle policies under different queue caps and arrival distributions.

Pong configuration reference: github.com/ishan-monie/advanced-ml-hw1. The implementation and reported runs are local.

GitHub username, repository and link: ZikunZheng1129 / mlds490-hw1 / https://github.com/ZikunZheng1129/mlds490-hw1